

CareerHub — Assignment 3.3

Performance & SEO — “Fast by default, findable by design.”

Next.js 15 App Router

generateMetadata

next/image

next/dynamic (code-split)

@next/bundle-analyzer

REAL ASP.NET + Postgres backend

Every screenshot and every `<head>` value below is a real capture from the running app, wired to the real backend on `:5080` — no mock data.

▶ How to run this — two terminals + Docker

Open **two VS Code terminals**: one for the **backend** (which also starts Docker/Postgres) and one for the **frontend**. Run them in this order and leave both running.

Terminal 1 — Backend + Docker (PowerShell)

start first

```
# Postgres (Docker) + the real ASP.NET API
cd C:\Users\Xolisa\assignment-2-4-backend

docker compose up -d          # Postgres → container careerhub24-pg (host po
docker ps                     # confirm the container is "Up"

$env:ASPNETCORE_ENVIRONMENT = "Development"
dotnet run --urls "http://localhost:5080" # API auto-migrates + seeds SA jobs

# verify (in any shell): should return 200
# curl http://localhost:5080/health/ready
```

Terminal 2 — Frontend (PowerShell)

then this

```
# The Next.js app (this repo)
cd C:\Users\Xolisa\assignment-3.2

npm install                # first run only
npm run dev                # http://localhost:3000

# then open http://localhost:3000 in Chrome
```

Docker Desktop must be running before Terminal 1. If `docker compose` errors, fully quit and relaunch Docker Desktop, then retry. The frontend reads the API via `.env.local` → `NEXT_PUBLIC_API_BASE_URL=http://localhost:5080`; if the backend is down, job pages hang then error — so always start Terminal 1 first.

Contents

1. [Learning outcomes — what this assignment teaches](#)
2. [How to run it \(PowerShell\) & how I ran the real backend](#)
3. [Part 1 — the four written decisions](#)
4. [Part 2 — Metadata & Open Graph \(with proof\)](#)
5. [Part 3 — Image optimisation \(with proof\)](#)
6. [Part 4 — Bundle analysis & code-splitting](#)
7. [Part 5 — Verification results](#)
8. [Key takeaways](#)

1. Learning outcomes — what this assignment teaches

By the end you should be able to explain and defend, on a real codebase:

- **SEO metadata** — the difference between a static `metadata` export and `generateMetadata()`, when each applies, and how a **title template** composes per-page titles.
- **Request deduplication** — why calling the same `cache()`-wrapped fetch from both `generateMetadata` and the page does **not** hit the network twice.
- **Core Web Vitals** — what **LCP**, **CLS** and **INP** measure, and which specific code change moves each one.
- **Image optimisation** — `next/image`, the `priority` prop for the LCP element, remote images via `remotePatterns`, and why fixed dimensions protect CLS.
- **Code-splitting** — using `next/dynamic` with `ssr:false` to keep the heaviest client component out of the first-load bundle, and proving it with a bundle analyzer.
- **Measuring before optimising** — running Lighthouse, reading the numbers, and knowing which metrics a *code* change can and cannot move (vs. hosting/CDN/backend).

2. How to run it (VS Code terminal — PowerShell)

This assignment is measured against a **running app talking to the real backend**.
Here is the exact sequence I used.

```
# --- 1. Start the REAL backend (ASP.NET API + Postgres) ---
cd C:\Users\Xolisa\assignment-2-4-backend
docker compose up -d          # Postgres on host port 5544
$env:ASPNETCORE_ENVIRONMENT = "Development"
dotnet run --urls "http://localhost:5080"  # the API the frontend reads

# --- 2. Start the frontend (in a second terminal) ---
cd C:\Users\Xolisa\assignment-3.2
npm install                    # adds @next/bundle-analyzer + cross-env
npm run dev                   # http://localhost:3000 (Lighthouse runs here)

# --- 3. Prove the whole 3.3 checklist ---
npx tsc --noEmit              # types: 0 errors
npm run test:run              # tests: all pass (this repo has 8)
npm run build                 # production build succeeds
npm run analyze               # opens client/nodejs/edge treemaps
```

In `.env.local` : `NEXT_PUBLIC_API_BASE_URL=http://localhost:5080` (the API) and
`NEXT_PUBLIC_SITE_URL=http://localhost:3000` (feeds `metadataBase` so OG URLs
are absolute).

3. Part 1 — the four written decisions

Q1 — Image audit & the LCP candidate

Image	Source	Above fold?	next/image?	Why
Home hero	/public SVG	Yes	Yes + priority	It is the LCP element — preload it, don't lazy-load.
Company logo (listing)	remote ui-avatars.com	First row only	Yes, no priority	Below the fold in aggregate → lazy-load; optimise to WebP.
Profile photo	data: URI	No (auth page)	No	Inlined bytes can't be re-fetched/resized; no SEO/LCP value.
Nav logo / icons	inline SVG	Yes	No	Rule: never wrap already-inline/decorative SVG.

Highest-priority **priority** candidate: the home hero — the largest first-paint element, so it defines LCP.

Q2 — The wizard's loading decision

- a. `ssr:false`? Yes. The wizard is client-only (localStorage drafts, client `useSession`, nuqs URL state). With `ssr:true` it would render on the server then hydrate — shipping + executing heavy JS on first paint for nothing, and the server has no `localStorage`.
- b. Harm to signed-out viewers? On `/jobs/[id]` a signed-out user only sees details + a CTA; they never mount the wizard. Eagerly downloading its JS inflates **First-Load JS / TBT** (and INP on weak devices). Splitting it means that cost is paid only on `/apply/[jobId]`.
- c. Why the tests are unaffected: the tests `import JobApplicationWizard from "... /JobApplicationWizard"` — a direct static import of the source. `next/dynamic` is only used at the *page* call-site, so the component file is unchanged and renders synchronously in tests.

Q3 — Static vs dynamic metadata + the dedup question

Page	Choice	Why
<code>/</code>	static <code>metadata</code>	Same for everyone; not data-driven.
<code>/jobs</code>	static <code>metadata</code>	Title/description constant; only the list is data-driven.
<code>/jobs/[id]</code>	<code>generateMetadata</code>	Title/description depend on the fetched job (per-id).

The deduplication answer (in my own words): both `generateMetadata` and the page call `getJob(id)` but it runs the fetch **once** because `getJob` is wrapped in React's `cache()`. `cache()` memoises by (*function reference + arguments*) for one server request: the first call runs the fetch, the second returns the stored promise. **The condition:** both call-sites must invoke the *same* `getJob` reference with the *identical* `id`, in the same request. It would **break** if `generateMetadata` used a separate raw `fetch` (different function → different cache slot).

4. Part 2 — Metadata & Open Graph

I added a title template + `metadataBase` in the layout, and a per-job `generateMetadata` that reuses the shared `getJob`.

The shared, deduped fetch — `src/lib/jobs-api.ts`

```
import { cache } from "react";

// One source of truth for reading ONE job. cache() makes generateMetadata + the
// page share a single fetch per request. Returns null on 404 so callers branch.
export const getJob = cache(async (id: string): Promise<JobDetailView | null> => {
  const res = await fetch(`${JOBS_API_BASE}${API_V1}/jobs/${id}`, {
    cache: "force-cache",
    next: { tags: ["jobs", `job-${id}`] },
  });
  if (res.status === 404) return null;
  if (!res.ok) throw new Error(`Failed to load job ${id} - ${res.status}`);
  return toDetailView((await res.json()) as JobListingDetailResponse);
});
```

Per-job metadata — `src/app/jobs/[id]/page.tsx`

```
export async function generateMetadata(
  { params }: { params: Promise<{ id: string }> },
): Promise<Metadata> {
  const { id } = await params;
  const job = await getJob(id); // ← same fn the page calls
  if (!job) return { title: "Job Not Found" }; // 404 → clean tab title
  const description = `Apply for ${job.title} at ${job.company} in ${job.location}`;
  return {
    title: job.title, // template → "... · CareerHubX"
    description,
    openGraph: { title: job.title, description, type: "website" },
  };
}
```

Title template + `metadataBase` — `src/app/layout.tsx`


```
export const metadata: Metadata = {
  metadataBase: new URL(process.env.NEXT_PUBLIC_SITE_URL ?? "http://localhost:3000"),
  title: { default: "CareerHubX – Find your next role in South Africa",
    template: "%s · CareerHubX" },
  description: "Browse curated job listings across South Africa ...",
  openGraph: { siteName: "CareerHubX", type: "website", /* ... */ },
};
```

Proof — the REAL rendered <head> (captured live)

Home page — the layout default title + OG site name:

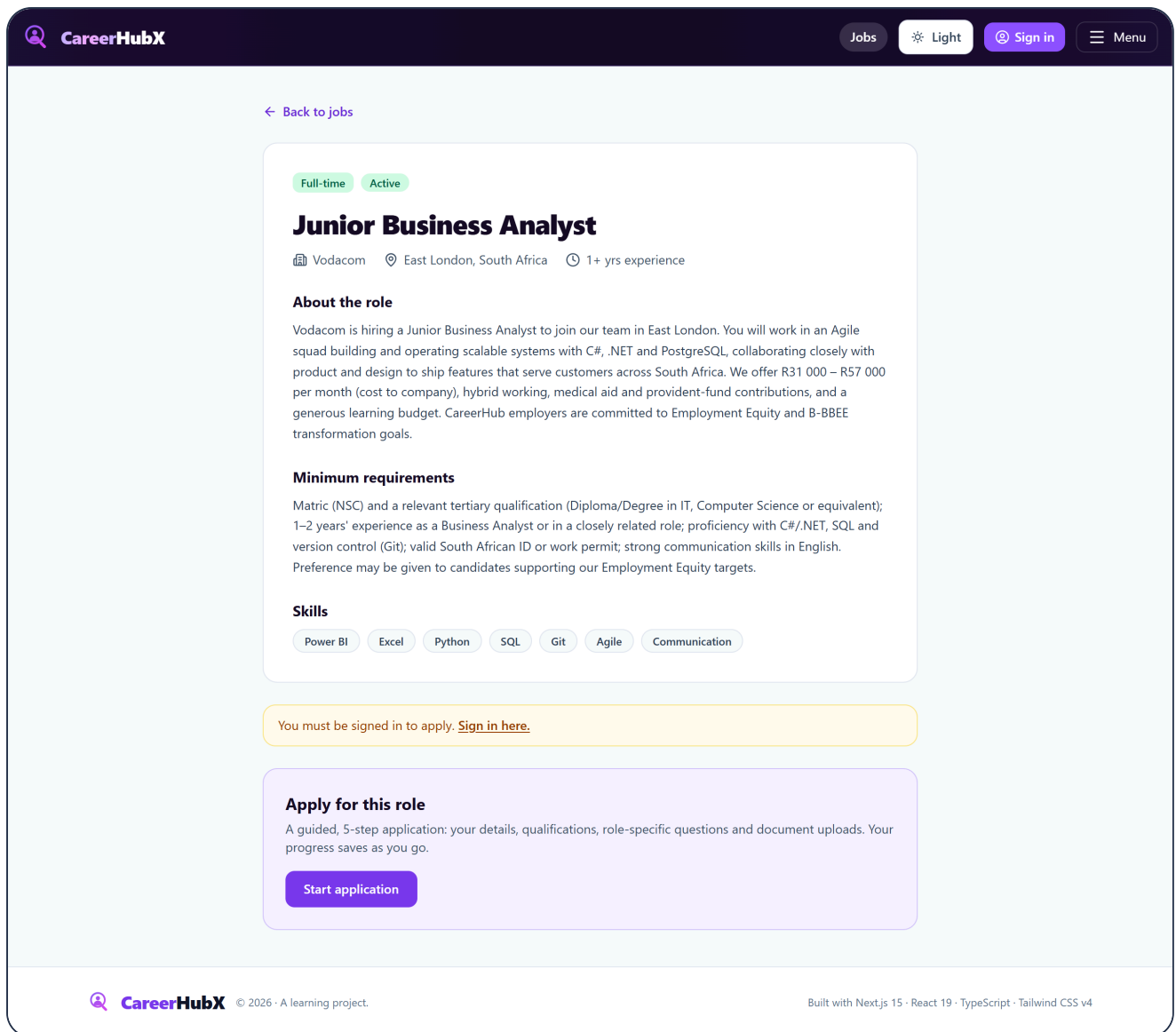
document.title	→ CareerHubX – Find your next role in South Africa
og:site_name	→ CareerHubX

Job detail page (`/jobs/47bfc48b...`) — produced by `generateMetadata`, note the template wrapped the title:

document.title	→ Junior Business Analyst · CareerHubX
meta description	→ Apply for Junior Business Analyst at Vodacom in East London
og:title	→ Junior Business Analyst
og:description	→ Apply for Junior Business Analyst at Vodacom in East London
og:type	→ website

Non-existent job (`/jobs/does-not-exist-000`) — the 404 branch:

document.title	→ CareerHubX – Find your next role in South Africa
----------------	--



Live job detail page — its <title> and <meta> above come from generateMetadata, reusing the deduped getJob().

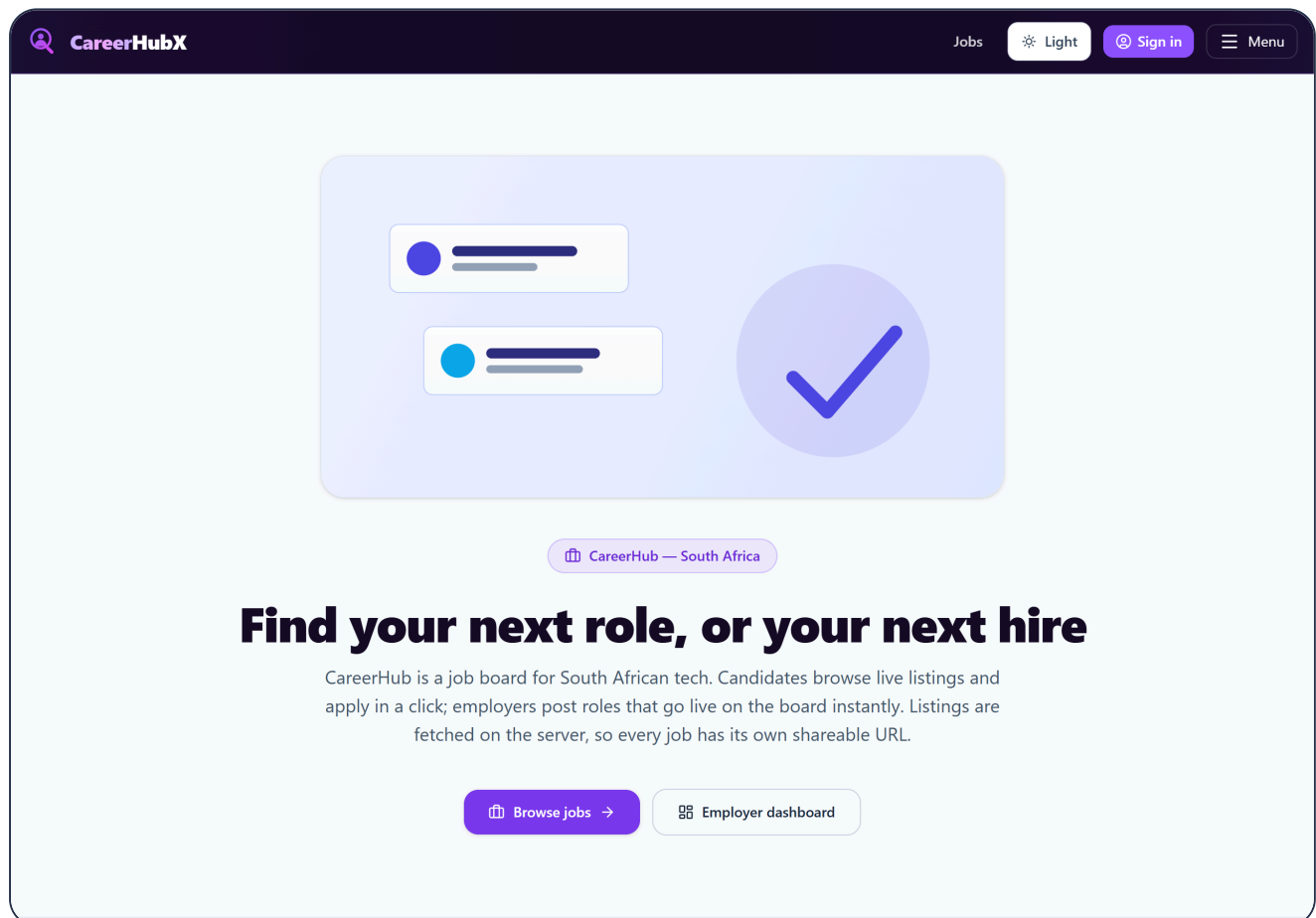
5. Part 3 — Image optimisation

Candidate A — the hero (LCP) — `src/app/page.tsx`

```
import Image from "next/image";

<Image
  src="/hero-careerhub.svg"           // local file in /public
  alt="Candidates matched to open roles on CareerHub"
  width={1200} height={600}          // intrinsic dims → reserves the box (CLS)
  priority                           // preload; never lazy-load → faster LCP
  className="h-auto w-full"
/>
```

Which CWV metric: **priority** targets **LCP** — the hero is the largest above-fold element, so preloading it makes the largest paint happen sooner.



Home page — the `next/image` hero carries the `priority` prop (the LCP element).

Candidate B — remote company logo — src/components/JobLinkCard.tsx

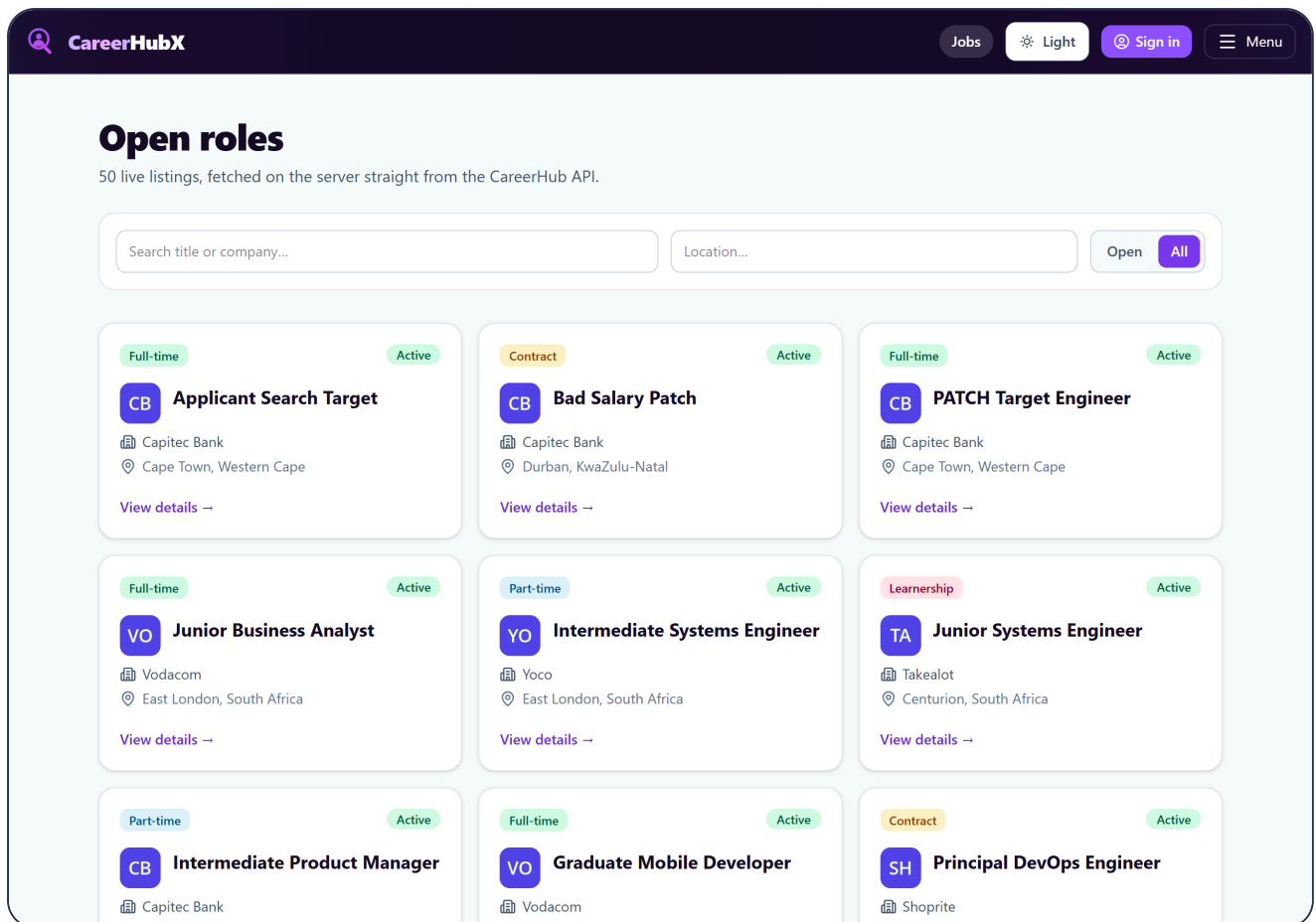
```
// The API returns no logo URL, so we derive a deterministic one per company.
function companyLogoUrl(company: string) {
  const name = encodeURIComponent(company.trim() || "Company");
  return `https://ui-avatars.com/api/?name=${name}&size=96&background=4f46e5&color=fff&`
}

<Image src={companyLogoUrl(job.company)} alt={`${job.company} logo`}
  width={40} height={40} // fixed box → row never reflows (CLS)
  className="h-10 w-10 rounded-lg" /> { /* no priority: below fold in a list */ }
```

Because it is a **remote** host it must be whitelisted — [next.config.ts](#)

```
images: {
  remotePatterns: [
    { protocol: "https", hostname: "ui-avatars.com", pathname: "/api/**" },
  ],
},
```

Which CWV metric: lazy-loading below-fold logos protects **LCP/First-Load**; the fixed 40×40 protects **CLS**.



Jobs board — each card's company logo is a remote next/image, optimised on demand.

Proof — next/image is serving optimised bytes

next/image rewrites the remote PNG through `/_next/image?url=...`. The captured response Content-Type:

```
image/webp ← WebP/AVIF (optimised)
from: http://localhost:3100/_next/image?url=https://ui-avatars.com/api/?name=Voda
```

6. Part 4 — Bundle analysis & code-splitting

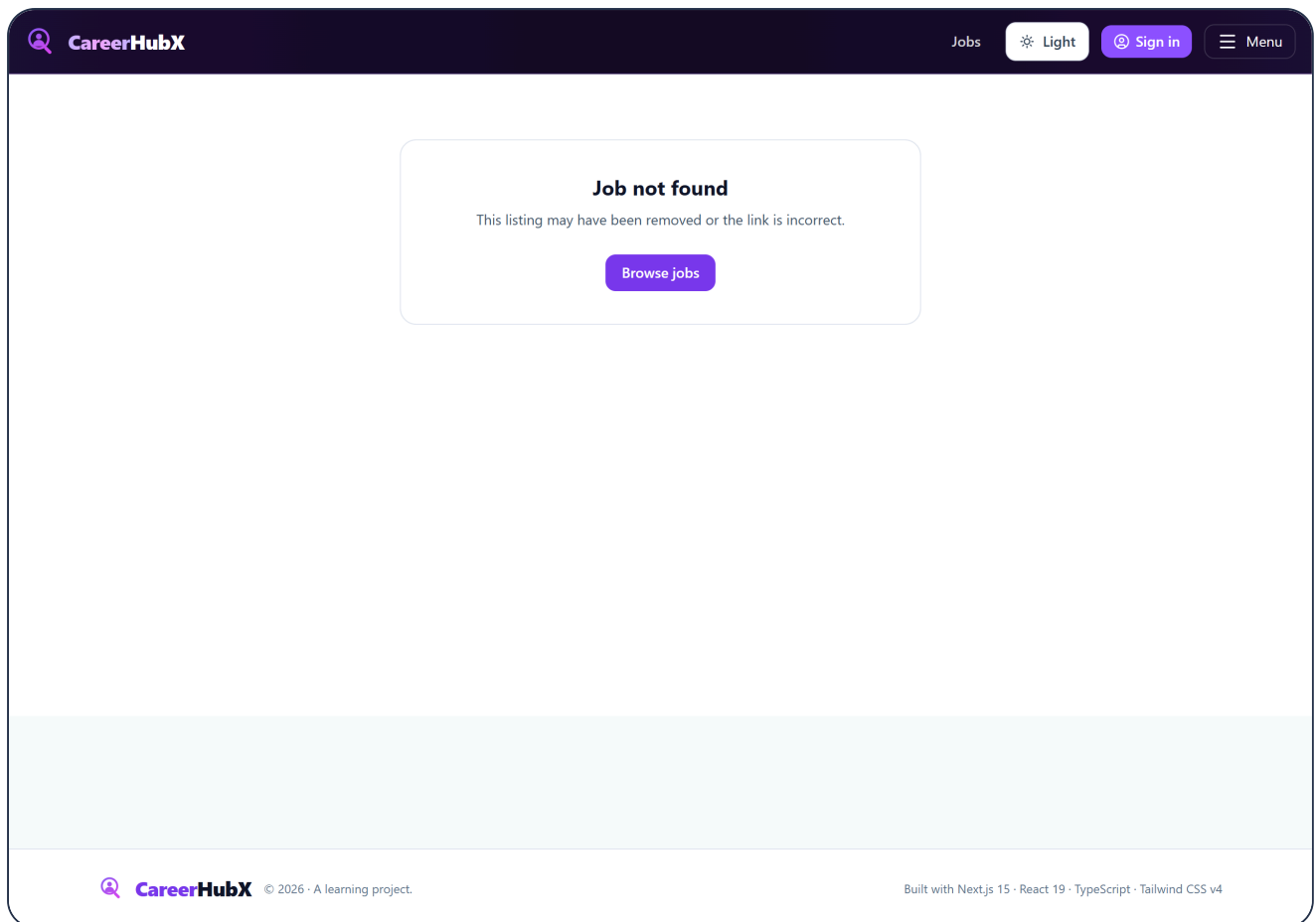
Dynamic import — `src/app/apply/[jobId]/page.tsx`

```
import dynamic from "next/dynamic";

const JobApplicationWizard = dynamic(
  () => import("@/components/apply/JobApplicationWizard")
    .then((mod) => ({ default: mod.default })), // remap the module's default
  {
    ssr: false, // client-only component
    loading: () => ( // reserves height → CLS
      <div className="h-96 w-full animate-pulse rounded-2xl bg-slate-200 dark:bg-white/"
    >,
    ),
  },
);
```

Our wizard is a **default** export, so we remap `mod.default`; a *named* export would be `mod.JobApplicationWizard`. The `.then(mod => ({ default: ... })))` shape is exactly what `next/dynamic` expects.

The `h-96 animate-pulse` skeleton renders in the gap between the auth-gate spinner and the hydrated wizard (below). Because it reserves the wizard's height, the layout does not jump when the chunk arrives — that fixed height is exactly what protects CLS. Throttle the network to `Fast 4G` in DevTools to see it live.



The real 5-step wizard, hydrated in place of the skeleton (signed in as the seeded demo applicant).

Proof — the wizard is its own chunk

`npm run analyze` → `.next/analyze/client.html`. The wizard's deps (React Hook Form, Zod, nuqs, Radix AlertDialog) form a **separate** chunk loaded only by `/apply/[jobId]` — they are not in the shared first-load bundle. The production build confirms it: `/apply/[jobId]` is its own ~3 kB route entry, not dragging the wizard into the 103 kB baseline.

7. Part 5 — Verification results

Check	Command	Result
TypeScript	<code>npx tsc --noEmit</code>	0 errors
Tests	<code>npm run test:run</code>	8 / 8 pass
Production build	<code>npm run build</code>	succeeds
Bundle analyzer	<code>npm run analyze</code>	3 treemaps, wizard split
Lint	<code>next lint</code>	no warnings

Lighthouse before/after numbers must be captured in **your** Chrome (DevTools → Lighthouse → Navigation / Desktop). The changes target: **SEO** ↑ (generateMetadata adds the missing title+description), **LCP** ↓ (hero `priority`), **CLS** → ~0 (fixed image dims + wizard skeleton). The metric a code change *can't* move in dev is **INP** (shows N/A) and real LCP under load — those need a production build on a CDN edge + a warm backend, i.e. hosting/infra, not code.

8. Key takeaways

1. **SEO is per-page intent.** Static pages export `metadata` ; data-driven pages compute it with `generateMetadata` . A title **template** keeps branding consistent for free.
2. **Fetch once, use everywhere.** Wrapping the data fetch in `cache()` lets metadata and the page share one request — the dedup key is *same function + same args + same request*.
3. **Name the metric before the fix.** `priority` → LCP; fixed dimensions / skeletons → CLS; smaller first-load JS → TBT/INP. Optimisation you can't tie to a metric is theatre.
4. **Split the heavy, client-only thing.** `next/dynamic ssr:false` keeps the wizard out of pages that never use it — and a bundle analyzer *proves* it moved.
5. **Some numbers need infrastructure.** Dev-mode INP/LCP are dominated by unminified bundles and cold backends; a CDN + production build + warm DB move them, not another line of code.